

// Jivops

ScyllaDB

Guía completa — 20 lecciones

Arquitectura shard-per-core · Rendimiento y diseño · Operación y
mantenimiento · Seguridad, monitoreo y migración

Contenido

Módulo 1: Fundamentos y arquitectura shard-per-core

Lección 01 — Qué es ScyllaDB y su relación con Cassandra

Lección 02 — Arquitectura shard-per-core con Seastar

Lección 03 — Anillo de tokens, vnodes y tablets

Lección 04 — Modelo de datos y drivers shard-aware

Lección 05 — Instalación, XFS e iotune

Módulo 2: Rendimiento y diseño de datos

Lección 06 — Caché propia de Scylla y E/S directa

Lección 07 — Scheduling groups y control del trabajo de fondo

Lección 08 — Particiones grandes y hot partitions

Lección 09 — Índices secundarios y vistas materializadas

Lección 10 — Consistencia ajustable y purga de tombstones (tombstone_gc)

Módulo 3: Operación y mantenimiento

Lección 11 — Estrategias de compactación y espacio libre

Lección 12 — Repair a nivel de fila

Lección 13 — Scylla Manager

Lección 14 — Escalar — añadir, retirar y reemplazar nodos

Lección 15 — Backups y snapshots

Módulo 4: Seguridad, monitoreo y migración

Lección 16 — Seguridad — autenticación, roles y TLS

Lección 17 — Monitoreo y desequilibrio entre shards

Lección 18 — Change Data Capture (CDC)

Lección 19 — Migración desde Cassandra y Alternator (API de DynamoDB)

Lección 20 — Proyecto final: ScyllaDB listo para producción

Módulo 1: Fundamentos y arquitectura shard-per-core

Lección 01 — Qué es ScyllaDB y su relación con Cassandra

Es una base de datos distribuida NoSQL reimplementada en C++ que mantiene compatibilidad con Cassandra (protocolo CQL y modelo de datos) y también ofrece una API compatible con DynamoDB, con una arquitectura sin JVM pensada para baja latencia y alto rendimiento por nodo.

Sin recolector de basura no hay pausas de GC que detengan el procesamiento de peticiones; la gestión de memoria es más directa y predecible, y la latencia en los percentiles altos mejora.

Lección 02 — Arquitectura shard-per-core con Seastar

Cada núcleo de CPU ejecuta un shard con su propia porción de memoria, cola de E/S y datos, sin compartir estado con otros núcleos (shared-nothing), sobre el framework asíncrono Seastar; así se evitan bloqueos y cambios de contexto, y la comunicación entre shards se hace por paso de mensajes.

Cada núcleo trabaja con sus propios datos sin esperar bloqueos de otros núcleos, evitando contenciones y cambios de contexto costosos, así que añadir núcleos aumenta el rendimiento casi de forma lineal.

Lección 03 — Anillo de tokens, vnodes y tablets

Por tokens (Murmur3 sobre la partition key) entre nodos y shards; los vnodes dividen el anillo en muchos rangos por nodo, y las versiones recientes introducen tablets, que reparten cada tabla en fragmentos gestionados dinámicamente para reequilibrar y escalar más rápido (su disponibilidad depende de la versión y de la configuración del keyspace).

Se pueden mover fragmentos concretos entre nodos sin recalcular rangos enteros del anillo, por lo que el reequilibrio es más gradual y automático y la carga se reparte de forma más fina.

Lección 04 — Modelo de datos y drivers shard-aware

El modelo es el de Cassandra (partition key, clustering, tablas por consulta); los drivers shard-aware calculan qué nodo y qué shard (núcleo) posee la partición y abren una conexión directa a él, evitando saltos internos entre núcleos y mejorando la latencia.

Evita que otro núcleo reciba la petición y tenga que reenviarla al núcleo propietario, ahorrando un salto interno y trabajo de coordinación, ya que la petición llega al shard que tiene los datos.

Lección 05 — Instalación, XFS e iotune

Linux con XFS, servidores dedicados y discos locales rápidos; `scylla_setup` configura el sistema y ejecuta `iotune`, que mide el rendimiento real del disco para ajustar el planificador de E/S de Scylla; no conviene compartir el servidor con otras cargas, porque Scylla asume el control de núcleos y memoria.

Scylla asume el control de los núcleos y la memoria asignados, y otras cargas generan interferencia; los picos de otras aplicaciones causan latencias impredecibles, porque el diseño depende de recursos dedicados.

Módulo 2: Rendimiento y diseño de datos

Lección 06 — Caché propia de Scylla y E/S directa

Gestiona su propia caché de filas y lee con E/S directa, sin depender de la caché de páginas de Linux, por lo que usa la mayor parte de la RAM del servidor; una caché bien dimensionada reduce las lecturas de disco, y una tasa de aciertos baja indica que el conjunto de trabajo no cabe.

Cada fallo de caché obliga a leer de disco, que es mucho más lento que la memoria, y suele indicar que el conjunto de trabajo no cabe; más memoria o mejor modelado reducen los fallos.

Lección 07 — Scheduling groups y control del trabajo de fondo

Separa las tareas en grupos de planificación con cuotas (shares): las consultas de usuario, la compactación, el streaming y el repair compiten con prioridades, y un controlador ajusta dinámicamente la compactación para que el trabajo de fondo no degrade la latencia de las peticiones.

El trabajo de fondo consume CPU y disco y, sin control, competiría con las peticiones; con cuotas se limita su impacto en los percentiles altos y el sistema sigue compactando sin dejar de atender consultas.

Lección 08 — Particiones grandes y hot partitions

Las particiones, filas o celdas muy grandes se registran en tablas del sistema como `system.large_partitions`, y `nodetool toppartitions` muestra las claves más activas; una hot partition concentra la carga en un solo shard, que se satura mientras los demás quedan ociosos, y se mitiga con buckets en la clave.

Una partición pertenece a un único shard y todo su tráfico lo atiende ese núcleo; no se puede repartir entre núcleos mientras sea la misma clave, así que el resto queda ocioso mientras ese satura.

Lección 09 — Índices secundarios y vistas materializadas

Se mantienen automáticamente a partir de la tabla base, lo que añade escrituras y consumo de recursos; permiten consultar por otras claves sin duplicar datos a mano, pero conviene medir su coste y vigilar el retraso en actualizar las vistas bajo carga alta.

Cada escritura en la tabla base debe generar también las actualizaciones correspondientes en la vista o el índice, multiplicando operaciones y tráfico entre nodos; hay que valorar si la consulta justifica ese coste.

Lección 10 — Consistencia ajustable y purga de tombstones (`tombstone_gc`)

Scylla mantiene consistencia ajustable (`ONE`, `QUORUM`, `LOCAL_QUORUM`...) como Cassandra; para evitar datos zombi, la opción `tombstone_gc` permite que las tombstones solo se purguen tras un repair (modo repair) en lugar de depender únicamente de `gc_grace_seconds`, siempre que la versión la soporte y el repair se ejecute con regularidad.

Asegura que todas las réplicas han recibido el borrado antes de eliminar la tombstone; si se purga antes, una réplica atrasada puede devolver el dato borrado, así que se vincula la purga a la convergencia de las réplicas.

Módulo 3: Operación y mantenimiento

Lección 11 — Estrategias de compactación y espacio libre

STCS fusiona SSTables de tamaño similar y favorece escrituras con mayor amplificación de espacio; LCS mantiene niveles con pocas SSTables por lectura a costa de más E/S; TWCS agrupa por ventanas de tiempo para series temporales con TTL; en algunas ediciones existe ICS, que reduce la amplificación de espacio; en todos los casos hay que dejar espacio libre en disco para compactar.

La compactación crea nuevas SSTables antes de eliminar las antiguas; sin espacio libre no puede completarse y se acumulan SSTables, con lo que las lecturas se ralentizan y las escrituras pueden fallar.

Lección 12 — Repair a nivel de fila

Trabaja a nivel de fila (repair basado en filas), comparando y transfiriendo solo lo que difiere, y es más eficiente que los árboles de Merkle clásicos; debe ejecutarse con regularidad (idealmente programado con Scylla Manager) dentro de `gc_grace_seconds` para mantener las réplicas convergentes.

Envía solo las filas que difieren entre réplicas y no toda la partición, con menos tráfico de red y menos lectura y escritura de disco, así que reduce el impacto sobre las peticiones de los clientes.

Lección 13 — Scylla Manager

Coordina tareas operativas del cluster mediante un agente en cada nodo: programa repairs de forma ordenada, hace backups a almacenamiento de objetos como S3, comprueba la salud del cluster y lleva el historial de las tareas, evitando depender de scripts manuales.

Coordina el orden y evita lanzar tareas pesadas a la vez en muchos nodos, mantiene historial y estado y avisa si algo falla, reduciendo el riesgo de que una tarea se olvide o falle sin que nadie lo note.

Lección 14 — Escalar — añadir, retirar y reemplazar nodos

Al añadir un nodo este recibe sus datos por streaming y, con vnodes, hay que ejecutar `nodetool cleanup` en los nodos antiguos para eliminar los datos que ya no les corresponden; con tablets el reequilibrio es automático según la versión; para retirar un nodo vivo se usa `decommission` y para uno muerto `replace` o `removenode`, siempre vigilando el estado del cluster.

El streaming consume red y disco y puede afectar a la latencia de las peticiones, y un fallo durante la operación deja el cluster en un estado intermedio; comprobar que todo está sano antes de continuar evita perder redundancia.

Lección 15 — Backups y snapshots

Un snapshot crea enlaces duros a las SSTables de forma casi instantánea en cada nodo y debe copiarse fuera del nodo; Scylla Manager puede enviar copias a almacenamiento de objetos y restaurarlas; la replicación no es un backup, porque un borrado accidental o una corrupción lógica se replican a todas las réplicas, y hay que probar las restauraciones.

Una copia que no se ha restaurado nunca puede estar incompleta o no ser utilizable; la prueba descubre problemas de procedimiento y mide cuánto tarda realmente la recuperación antes de un incidente real.

Módulo 4: Seguridad, monitoreo y migración

Lección 16 — Seguridad — autenticación, roles y TLS

Hay que activar la autenticación y la autorización por roles (no vienen exigidas por defecto), cambiar o desactivar la cuenta de administración por defecto, replicar el keyspace de autenticación, cifrar con TLS el tráfico cliente-nodo e internodo, y aislar la red del cluster; el cifrado en reposo está disponible según la edición, y si no, se recurre a cifrado de disco.

Las credenciales por defecto son públicas y cualquiera podría usarlas para obtener control total; los escaneos buscan servicios con usuarios por defecto, y una cuenta propia con contraseña fuerte reduce el riesgo.

Lección 17 — Monitoreo y desequilibrio entre shards

Scylla Monitoring Stack (Prometheus, Grafana y Alertmanager) ofrece paneles y alertas; conviene vigilar latencias por percentil, tasa de aciertos de caché, compactaciones pendientes, timeouts, reactor stalls y, sobre todo, el desequilibrio entre shards: un núcleo saturado con otros ociosos suele indicar una hot partition o un cliente que no usa drivers shard-aware.

Un núcleo al máximo con el resto ociosos produce una media baja, pero la latencia la determina el shard saturado y no la media, así que hay que mirar las métricas por shard.

Lección 18 — Change Data Capture (CDC)

Registra los cambios de una tabla (inserciones, actualizaciones, borrados, con imágenes previas y posteriores opcionales) en una tabla de log asociada que se consume desde aplicaciones o conectores como Kafka; los registros tienen un TTL (por defecto de 24 horas), así que los consumidores deben leer a tiempo.

El log tiene una retención limitada y los cambios antiguos se eliminan; si el consumidor se retrasa más que el TTL pierde cambios, así que hay que dimensionar el TTL y vigilar el retraso del consumidor.

Lección 19 — Migración desde Cassandra y Alternator (API de DynamoDB)

Desde Cassandra se puede migrar con herramientas como sstableloader o Scylla Migrator (basado en Spark), normalmente con doble escritura y validación previa; Alternator ofrece una API compatible con DynamoDB para mover aplicaciones; la compatibilidad no es absoluta, por lo que hay que probar el comportamiento, las funciones usadas y el rendimiento.

Pueden existir diferencias de comportamiento o funciones no soportadas que solo aparecen con la carga real; se detectan problemas de rendimiento y de datos antes de afectar a producción y se ensaya el plan de vuelta atrás.

Lección 20 — Proyecto final: ScyllaDB listo para producción

Hardware dedicado con XFS y discos locales, drivers shard-aware, tablas diseñadas por consulta con particiones acotadas, RF 3 por DC con LOCAL_QUORUM, repair y backups programados con Scylla Manager y restauraciones probadas, autenticación y TLS, y monitoreo por shard de latencias, caché y compactaciones.

Intenta mencionar al menos dos pilares: hardware dedicado con drivers shard-aware, tablas diseñadas por consulta con particiones acotadas, RF 3 por DC con LOCAL_QUORUM, repair y backups con Scylla

Manager, o autenticación, TLS y monitoreo por shard.